

好，下面给你一份**适配你那份 PPT 的完整中文演讲稿**。
你可以直接照着念，也可以按自己的习惯稍微改一改语气。

【第 1 页】GaussianUpdate：持续更新的 3D 高斯场 ——用于动态场景的实时渲染与变化追踪

大家好，我是 XXX，今天想和大家分享一篇我最近看的论文，名字叫 **GaussianUpdate**。
它解决的问题很有意思：现实世界的三维场景一直在变，我们怎么让 3D 模型一边“跟着环境更新”，一边还能保持**高质量、实时渲染**，而且不把以前学到的东西忘掉。
接下来大概 10 分钟，我会用比较直观的方式带大家了解这篇工作的核心思想和亮点。

【第 2 页】研究背景：为什么要“更新”3D 场景模型？

首先，我们看一下研究背景。
在很多应用里，比如 AR/VR、机器人导航、自动驾驶，我们都希望有一个**三维数字世界**，可以从任意视角生成逼真的图像。
但现实世界的一个特点是：**它不是静止的**。
像右边这几种情况：

- 白天到晚上，光照变了；
 - 房间里的桌子椅子被挪走或新放了东西；
 - 车门打开、关闭，车子开进来、开出去；
- 这些都是非常常见的变化。

传统的 NeRF 这类方法有两个问题：

1. 它们通常假设场景是静态的，一旦环境变了，就要重新训练一遍模型，成本很高；
2. 如果我们只拿“最新的数据”去继续训练，很容易出现一个问题：**灾难性遗忘**，也就是模型把以前学到的场景细节忘了。

所以这篇论文想要做的是：

让三维场景模型像人一样，能一边学习新情况，一边保留对过去的记忆。

【第 3 页】核心思想：GaussianUpdate 在做什么？（图 1）

这页是这篇论文最直观的一张图，我们从左往右看。

最左边是一个已经训练好的 **3D 高斯场**，可以理解为场景的三维点云，只不过每个点不是一个小球，而是一个带颜色、大小、方向的「高斯云团」。

这些高斯通过高斯 splatting 的方式可以实时渲染图像。

当时间往后走，我们会获得两类数据：

- 一类是**当前新采集的图像**；
- 另一类叫 **replay data**，是从旧模型渲染出来的历史视角数据，可以看成模型自己“回忆”过去。

GaussianUpdate 做的事情，就是用这两类数据一起对模型进行更新：

- 一方面通过当前数据，让模型适应新的场景变化；
- 另一方面通过回放数据，让模型不要忘记之前的样子。

更新之后的高斯场，就像图中间那块，既能表示新的状态，又保留了旧的信息。

右边可以看到，它还能把 **不同时间点的几何布局变化** 可视化出来，比如车门从关着变成打开，这部分高斯会发生显式的变化。

【第 4 页】场景变化类型：光照、消失、新增

要想设计一个好用的更新机制，首先要搞清楚：场景到底是**怎么变的**？

论文把变化分成了三类：

1. 光照变化：

比如同一个院子，晴天、阴天、晚上，整体亮度、阴影、颜色都会变，这其实几何没变，只是“打光”变了。

2. 物体消失：

比如桌子被搬走，车开出停车位，或者车门从打开变成关上，本质上是某些高斯点不该再出现了。

3. 物体新增：

比如新放了一个箱子，新停一辆车，或者车门从关闭变成打开，这些都需要在模型里创建新的高斯点。

这三类变化在现实中非常常见。GaussianUpdate 的一个亮点就是：

它不是把所有变化混在一起优化，而是**显式区分这三类变化**，然后分别用不同的策略来处理。

【第 5 页】系统整体结构（图 2）

接下来我们看一下整体系统结构，这张图信息很多，我简单从左到右给大家扫一遍。

左边是**已有的 3D 高斯模型**，同时还有一个 **Visibility Mask（可见性掩码）** 和一个 **4D 哈希表**，用来存跟时间相关的外观信息。

上面这条链路是渲染过程：

- 给定时间 t 和相机视角，系统会查询当前时刻哪些高斯是可见的；
- 通过 4D 哈希+MLP 算一组“外观增量”，也就是 ΔY 、 ΔS ，表示颜色和尺寸等怎么因为时间而改变；
- 再把这些高斯 splatting 到屏幕上，生成对应视角的图像。

中间这一圈是“变化检测”：

- 模型会比较当前图像和用历史模型渲染出的图像，配合 SAM 等工具找出哪些区域基本没变，哪些区域可能发生了几何变化；
- 这会生成一个「不变区域」掩码，后面会用来指导外观更新。

最下面这一排就是论文中提到的 **三阶段更新**：

- 第一阶段主攻光照；
- 第二阶段主攻几何新增和删除；
- 第三阶段做整体的精修和剪枝。

最右边则是更新后的模型，它既包含新的 3D 高斯，也包含新的可见性掩码和外观模型。

【第 6 页】三阶段更新策略：外观 + 几何 + 细化

这三阶段是这篇论文另一个很重要的设计亮点，我们逐个看。

第一阶段：外观更新

- 在这一阶段，作者只在“布局不变”的区域里更新外观，也就是**只改光照，不碰几何**。
- 做法是利用 4D 哈希+一个小 MLP，根据高斯的位置和时间 t ，预测这个点在当前时间的颜色/亮度增量。
- 这样可以很好地拟合早晚不同、阴晴不同导致的全局光照变化，又不会因为几何没对齐而乱。

第二阶段：几何布局更新

- 先用 COLMAP 从新图像里算出稀疏点云，在新物体的地方创建新的高斯点。
- 对老的高斯点引入一个“移除因子”，训练时如果发现某些高斯总是解释不了当前图像，就会把它们的权重推向 0，相当于学会“这块东西已经不存在了”。
- 中间还有 DBSCAN 之类的聚类，用来过滤噪声点，避免乱删。

第三阶段：联合细化 + 剪枝

- 这一步再把外观模型和几何一起细调，修掉删点之后的一些伪影。
- 同时根据“重要性评分”删掉贡献很小的高斯，模型变得更加精简。

整体思路可以总结成一句话：

先分开处理光照和几何，再整体微调，既稳定又高效。

【第 7 页】持续学习：可见性池与生成式回放

光有更新还不够，我们还得防止模型**忘记过去**。

GaussianUpdate 在这里用了两个小技巧：

第一个是 **可见性池 (Visibility Pool)**：

- 当我们“删除”某个高斯时，其实没把它从内存里抹掉，只是把它标记为“当前时刻不可见”；
- 这样如果我们想恢复过去的场景，只要把对应时间的高斯重新标记为可见就行了，相当于模型里自带一个“历史版本控制”。

第二个是 **生成式回放 (Generative Replay)**：

- 传统的持续学习 often 会存一部分旧数据，用来防遗忘，但这样需要很多存储空间；
- 这篇论文的做法是：只保存一些旧的相机姿态，每次训练新时间点时，用当前模型重新渲染这些旧视角，把这些生成图像当作“历史样本”；
- 这样在训练的时候，模型一边看新数据，一边也在“复习”旧数据，就不容易把过去的记忆忘掉。

这两个设计加在一起，就让 GaussianUpdate 成为了一个**既会学习、又会回忆**的 3D 场景模型。

【第 8 页】实验结果：可视化对比与性能（图 4）

接下来简单看下实验结果。这里展示的是 WAT 数据集上的可视化。

每一组两行对应同一个场景，上面是当前时刻的图像，下面是过去时刻的图像；
每一列是不同方法：Baseline、CLNeRF、4DGS、Ours、UB、以及真实图像 GT。

可以看到红框标出来的这些细节：

- Baseline 很容易过拟合到当前数据，过去的样子会画错；
- CLNeRF 虽然能保留一些记忆，但细节纹理和亮度都有差距；
- 我们的方法在很多细节位置上都更接近 Ground Truth，而且过去和现在都能画得比较好。

在定量指标上，这篇论文的 PSNR、SSIM 大部分场景都比对手高；
更重要的是，由于使用 3D 高斯 splatting，**推理速度能达到 30 多 FPS**，是真正意义上的实时渲染，而 NeRF 类方法往往不到 1 FPS。

所以整体评价就是：**质量好、速度快、记忆力也不错。**

【第 9 页】总结：为什么这篇工作值得关注？

最后我们做一个小结。

我个人觉得这篇工作的亮点可以归纳成四点：

1. **表征层面**：把 3D Gaussian 和持续学习结合起来，是一个很自然又很新鲜的组合；
2. **算法设计层面**：三阶段更新，把光照变化、几何新增、几何消失分别建模，结构清晰，易于扩展；
3. **持续学习层面**：可见性池 + 生成式回放，在不存大量图像的情况下，解决了灾难性遗忘问题；
4. **工程效果层面**：在真实数据集上实现了高质量且实时的新视角渲染，离实际应用又近了一步。

对我们大二同学来说，这篇论文还提供了一个挺好的“范式”：

- 一方面用显式几何表示保证效率和可解释性；
 - 另一方面用神经网络和持续学习的思想来做模型更新。
- 将来如果大家做 AR、机器人、数字孪生相关的项目，这类方法都是可以借鉴的思路。

【第 10 页】Q&A 与讨论

我的分享大致就到这里，谢谢大家的聆听。

留两个小问题，大家可以思考一下：

1. 如果场景里有很多镜子、玻璃、水面这样强反射、半透明的物体，GaussianUpdate 这种基于高斯的表示还好用吗？
2. 如果把这个方法部署到一台真正跑在室外的机器人或者 AR 眼镜上，还会遇到哪些额外的工程问题？

欢迎大家提问或者一起讨论。

如果你愿意，我还能再给你做一个**超精简版背诵稿**（每页 2-3 句话），方便你临场忘词时快速想起来要讲什么。